

컴퓨터 비전(AI응용)

이미지 생성형 AI

- 경쟁적(GAN), 구조적(DCGAN), 확률적(VAE) 접근을 통해 데이터 생성의 핵심 메커니즘과 모델 간 차이를 이해한다.
- Generator-Discriminator, Encoder-Decoder 구조와 손실 함수의 역할을 익히고, 실제 이미지 생성 모델을 설계·학습할 수 있는 능력을 기른다.
- 잠재 공간(Latent Space)을 활용해 새로운 데이터나 스타일을 생성·변형하며, 생성 모델을 활용한 AI 창의 응용 역량을 향상시킴.

Key Question

"AI는 새로운 이미지를
만들어낼 수 있을까?"

 기존의 AI (Discriminative)

✓ 분류 (Classification): "이게 고양이인가?"

✓ 검출 (Detection): "고양이가 어디에 있나?"

| 이미지를 보고 판단(Decision)하는 것에 집중

 생성형 AI (Generative)

★ 생성 (Generation): "없는 고양이를 그려라"

★ 합성 (Synthesis): "사람 얼굴을 만화처럼"

| 판단하지 않고 창조(Creation)하는 것에 집중



“

Definition

생성형 모델은 "데이터의 분포를 학습하여, 존재하지 않던 새로운 데이터를 만들어내는 모델"입니다.

? 왜 '확률'인가?

✓ 정답이 하나가 아님

숫자 '3'을 써도 사람마다 모양, 굵기, 기울기가 다름

✓ 이미지는 분포(Distribution)다

세상의 모든 고양이 사진은 어떤 확률 분포 안에 존재함

🎯 우리의 목표

"데이터가 만들어졌을 가능성 공간(Latent Space)을 학습하자"

이 공간에서 점 하나를 뽑으면(=Sampling), 새로운 이미지가 된다.

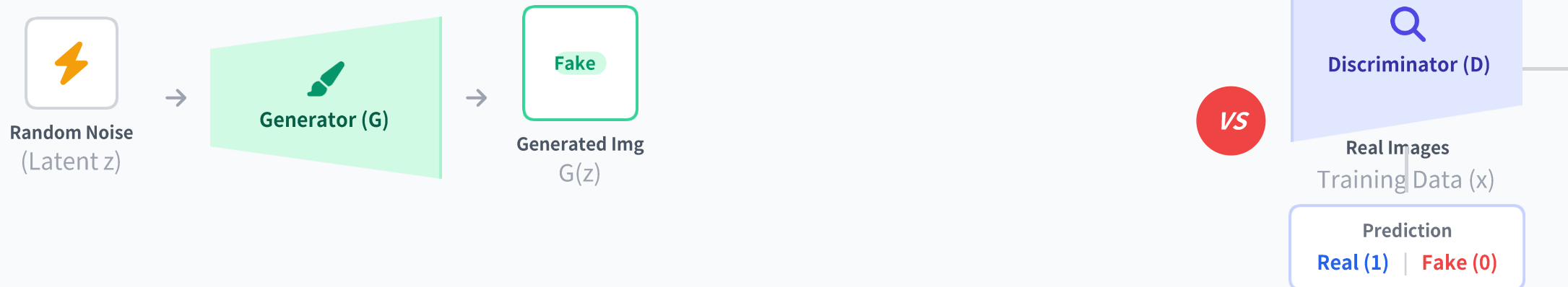
Latent Space Concept



GAN(Generative Adversarial Network)

두 개의 신경망(생성자와 판별자)이 서로 **경쟁(Adversarial)**하며 학습하여 진짜 같은 데이터를 생성하는 모델

Adversarial Training Loop



Key Players



Generator (G)

"경찰을 속일 만큼 완벽한
위조지폐를 만들겠다"



Discriminator (D)

"진짜 돈과 위조지폐를
완벽하게 구별해내겠다"

Minimax Game (게임 이론)

제로섬 게임(Zero-Sum Game)과 유사합니다.
G는 D가 실수하도록 유도하고(Maximize Error),
D는 자신의 판별 정확도를 높이려 합니다(Minimize Error).
이 경쟁의 균형점(Nash Equilibrium)에서 고품질 이미지가 생성됩니다.



- GAN 생성적 적대 신경망(Generative Adversarial Network)
 - 2014년 처음 제안된 생성 모델이다.
- **생성기(Generator)**와 **판별기(Discriminator)**라는 두 개의 신경망이 경쟁적으로 학습
 - 생성기는 진짜처럼 보이는 가짜 데이터를 생성하고,
 - 판별기는 입력된 데이터가 진짜인지 가짜인지 구별하는 역할을 한다.
- 이 두 모델은 **적대적 학습(adversarial learning)** 구조를 이루며 서로 발전한다.

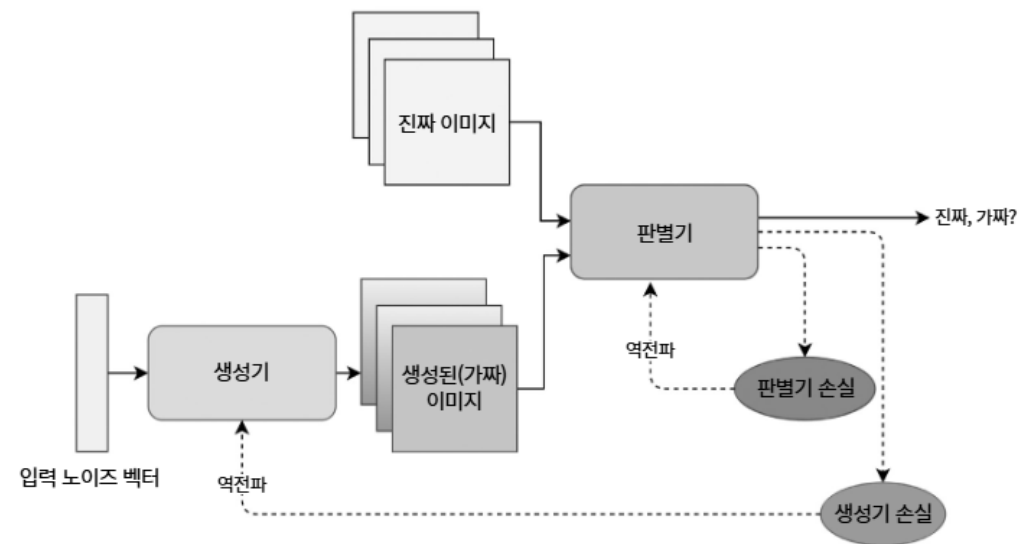


그림 8.1 GAN 구조

♟ 미니맥스 게임 (Minimax Game)

두 모델이 서로 반대되는 목적을 가지고 경쟁하며 학습하는 구조입니다.



Generator (G)

속이려는 자: D가 실수하도록 유도 (Minimize D's Success)



Discriminator (D)

구별하려는 자: 진짜와 가짜를 정확히 분류 (Maximize Accuracy)

⚠ 학습의 특징 및 대표 문제

⚡ 불안정한 학습 (Instability)

Loss가 줄어들지 않고 계속 진동(Oscillation)하는 현상이 빈번함.

🖨 모드 붕괴 (Mode Collapse)

G가 다양성을 잃고, D를 속이기 쉬운 **특정 이미지만 반복 생성**함.

Adversarial Loop



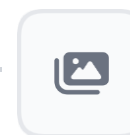
Generator

Fake Image



Discriminator

Real Image



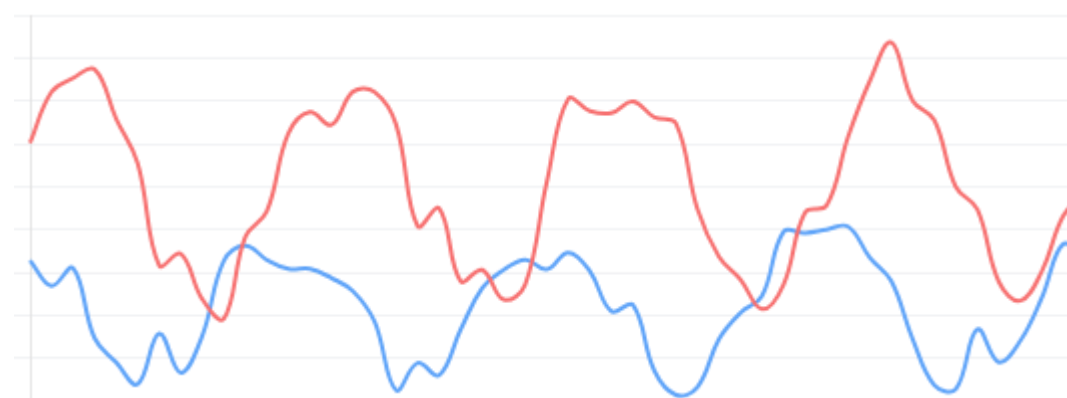
Dataset

× 속임수 실패

판별 성공 ✓

Loss Signal (Backpropagation)

학습 곡선 예시 (Loss Fluctuation)

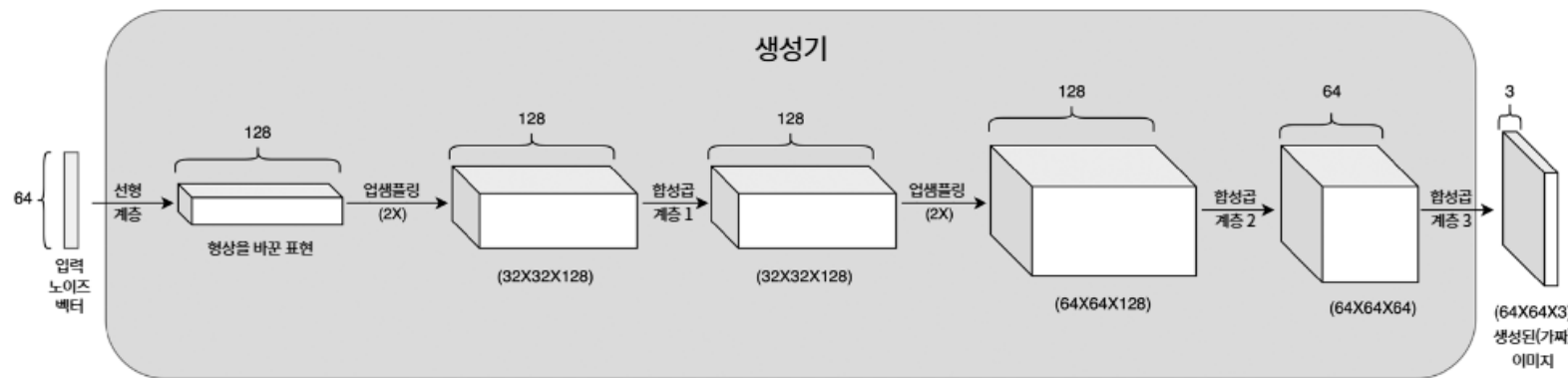


- 판별기의 목적은 진짜 이미지를 진짜로, 가짜 이미지를 가짜로 정확하게 분류하는 것이다.
- 생성기의 목적은 판별기를 속여 가짜 이미지를 진짜처럼 보이게 만드는 것이다.
- $D(x)$ 는 진짜 이미지에 대한 판별기의 출력, $D(G(z))$ 는 가짜 이미지에 대한 판별기의 출력이다.
- 학습은 교차 최적화(Alternating Optimization) 방식으로 진행
- 두 네트워크는 서로 반대되는 목적을 가지므로, 전체 학습은 “최소-최대 게임(minimax two-player game)” 형태로 표현된다.

$$L_D = -[\log D(x) + \log(1 - D(G(z)))]$$

$$L_G = -\log D(G(z))$$

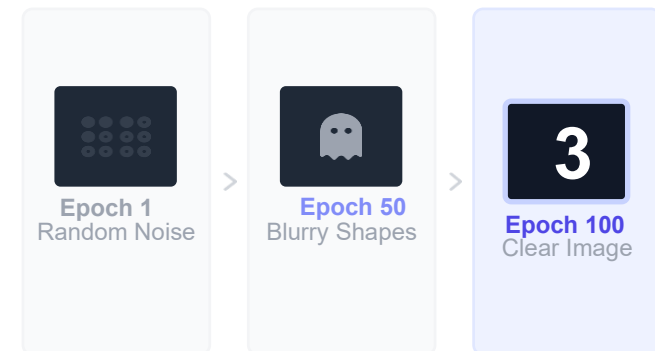
$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$



업샘플링: 가장 가까운 이웃 탐색 전략 기반으로 공간 차원을 따라 업샘플링
 합성곱 계층 1: in_channels=128, out_channels=128, kernel_size=3, stride=1, padding=ON
 합성곱 계층 2: in_channels=128, out_channels=64, kernel_size=3, stride=1, padding=ON
 합성곱 계층 3: in_channels=64, out_channels=3, kernel_size=3, stride=1, padding=ON

그림 8.2 DCGAN 생성기 모델 아키텍처

Epoch별 생성 결과 변화



- 입력: 64차원 랜덤 노이즈 벡터를 선형 계층을 통해 **16×16×128 특징 맵**으로 변환함.
 - 업샘플링 1단계: **16×16 → 32×32**, 합성곱 계층 사용.
 - 업샘플링 2단계: **32×32 → 64×64**, 추가 합성곱 계층 적용.
 - 각 단계마다 **3×3 커널, 128채널**을 사용하며, 점차 해상도를 높임.
 - 마지막에 **2차원 합성곱 계층**을 거쳐 **64×64 RGB 이미지** 생성.
- **핵심:** DCGAN은 노이즈로부터 점진적으로 해상도를 높이며 현실적인 이미지를 생성하는 구조임.

DCGAN 구현(판별기)

- 입력: **64×64 RGB 이미지(3채널)** 을 받음.
- 첫 합성곱 계층: 커널 크기 **3×3**, 스트라이드 2, **64채널** 특징 맵 생성.

이후 계층:

- 두 번째 합성곱 → **128채널**, 특징 맵 크기 **32×32 → 16×16**
- 세 번째 합성곱 → **256채널**, 특징 맵 크기 **16×16 → 8×8**
- 네 번째 합성곱 → **512채널**, 특징 맵 크기 **8×8 → 4×4**

- 마지막 단계

전결합층(Fully Connected Layer)으로 연결

시그모이드(Sigmoid) 함수를 통해

이미지가 진짜(1)인지 가짜(0)인지 판별.

- **핵심**: 판별기는 합성곱과 다운샘플링을 반복하며 입력 이미지의 특징을 추출하고,
최종적으로 **진짜/가짜 확률**을 출력하는 **이진 분류기** 역할을 수행함.

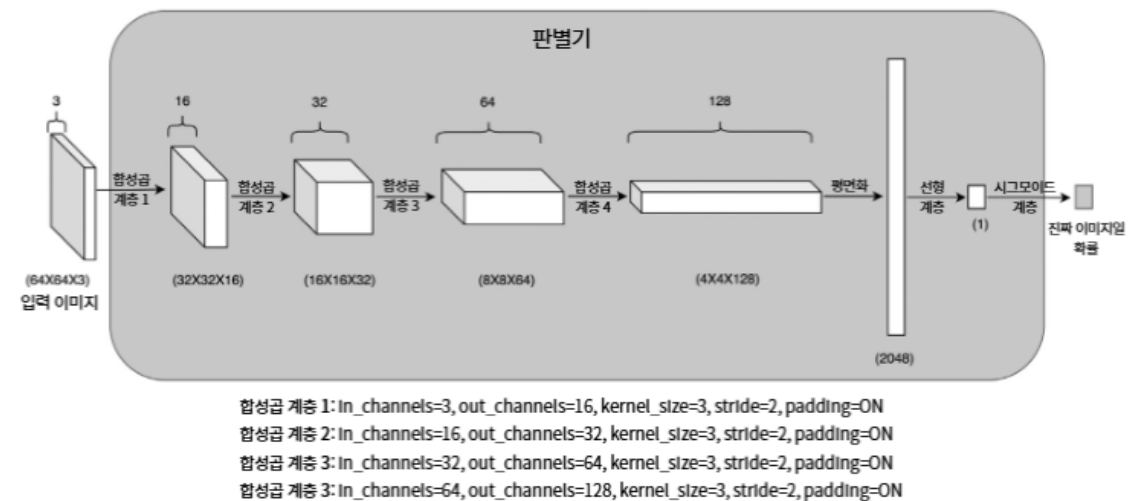


그림 8.3 DCGAN 판별기 모델 아키텍처

모델별 특성

- GAN의 다양한 변형은 손실 함수, 최적화 전략, 네트워크 구조에 따라 다름.
- SRGAN: 저해상도 이미지를 고해상도로 변환(초해상도).
- **CycleGAN**: 두 개의 생성기를 사용해 이미지 간 스타일 변환 수행.
- LSGAN: 일반적인 크로스엔트로피 손실 대신 **평균 제곱 오차(MSE)** 를 사용해 학습 안정성 향상.

Pix2Pix GAN

- 단순한 이미지 생성이 아니라 **이미지→이미지 변환(image-to-image translation)** 작업에 특화된 모델
- 스타일 전이, 색 변환, 스케치→사진 변환 등 **Layer 간 mapping 학습**을 수행.

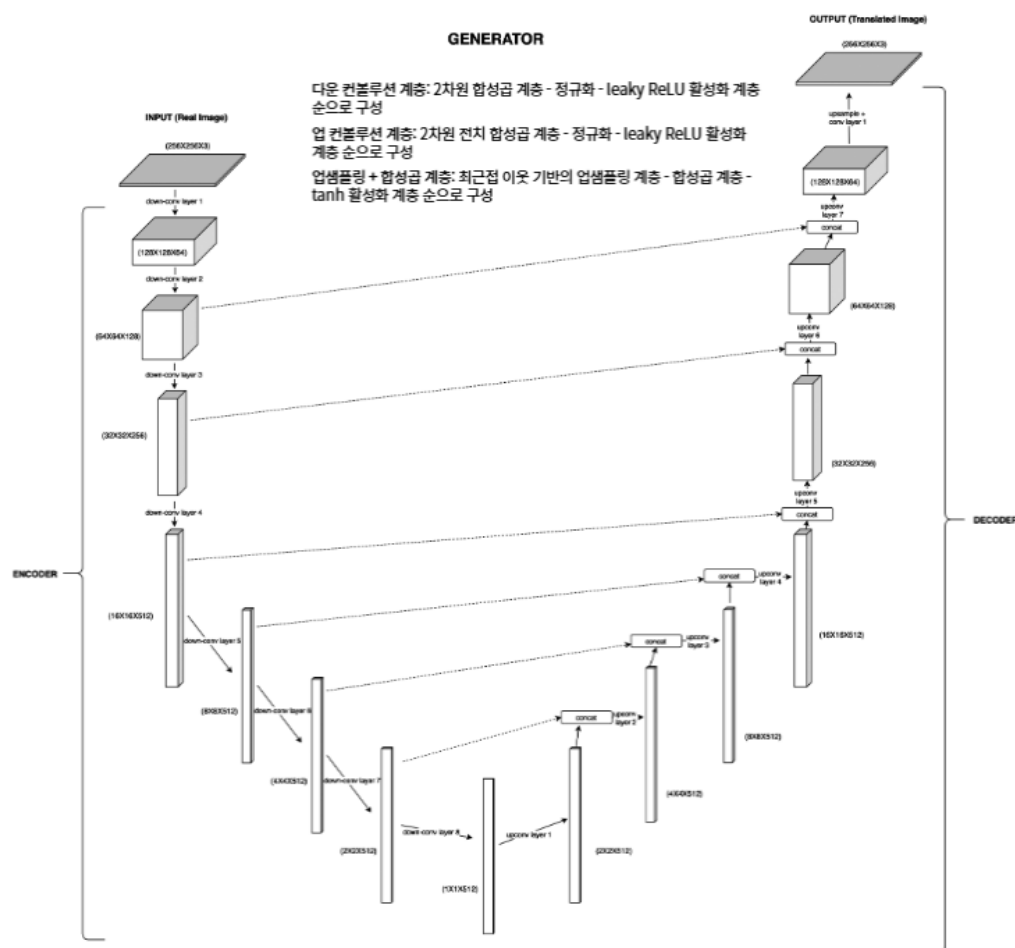


그림 8.7 Pix2Pix 생성기 모델 아키텍처

1.기본 구조: Encoder-Decoder 형태 (U-Net 기반).

2.Encoder (Downsampling):

1. 8단계 2차원 합성곱(conv2D) 계층으로 구성.
2. 각 계층: 합성곱 → 정규화 → Leaky ReLU 활성화.
3. 입력 이미지를 점진적으로 축소하며 특징 추출.

3.Decoder (Upsampling):

1. 업샘플링 + 합성곱 계층으로 구성.
2. 각 단계에서 skip connection으로 Encoder 출력과 연결(concat).
3. 활성화 함수: ReLU, 마지막 단계는 **tanh**.

4.출력:

1. 입력 이미지와 동일 크기의 변환 이미지(예: 256×256×3) 생성.
2. 이미지 간 변환(예: 흑백→컬러, 스케치→사진 등)에 활용.

→ **핵심:** Encoder로 압축된 특징을 Decoder로 복원하며, skip connection을 통해 세밀한 구조를 유지하는 **U-Net형 생성기**임.

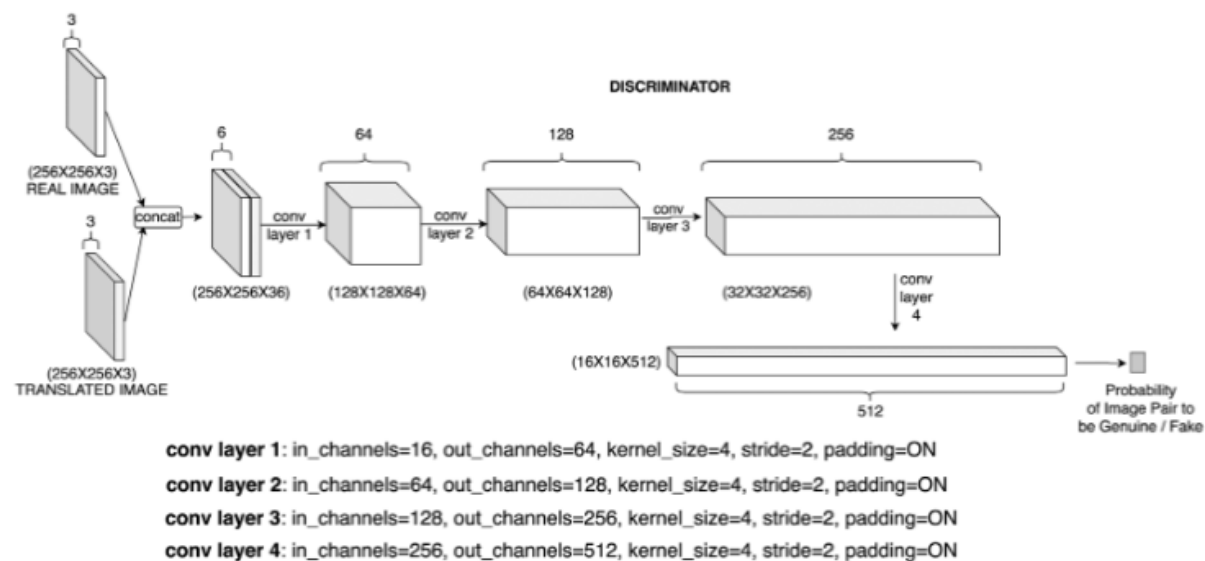


그림 8.8 Pix2Pix 판별기 모델 아키텍처

1.구조: PatchGAN 기반 CNN 판별기.

2.입력: 실제 이미지와 생성된 이미지를 나란히 결합(concat)하여 입력.

3.합성곱 계층:

- 여러 층의 conv + BatchNorm + Leaky ReLU 구성. 입력 이미지를 70×70 크기의 작은 패치 단위(patch) 로 구분해 판별.

4. 출력:

각 패치별로 진짜/가짜 확률을 예측 (전체 평균이 최종 판별 결과).

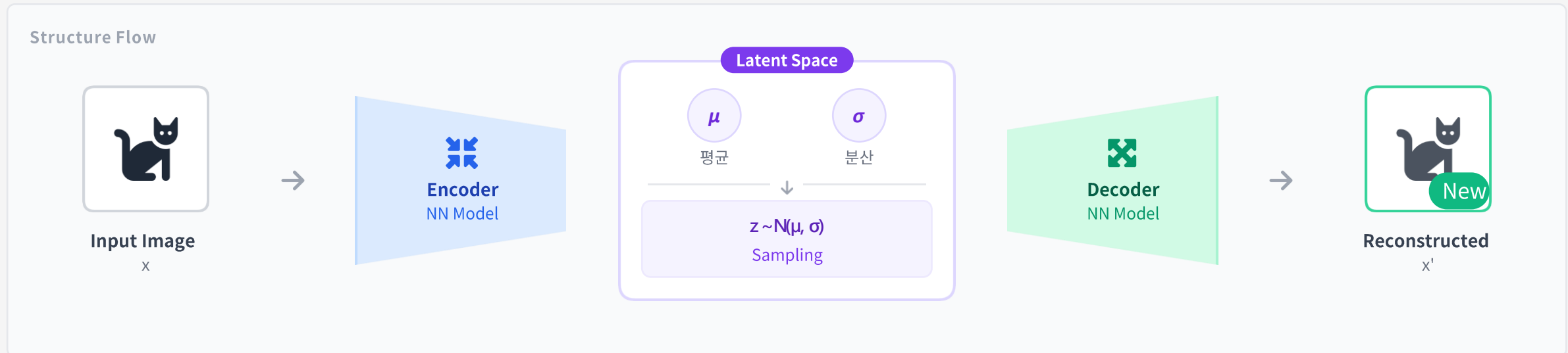
→ 핵심

PatchGAN은 이미지 전체가 아니라 국소 패치 단위로 진위 판단을 수행하여 세밀한 질감(realistic texture) 을 효과적으로 학습시킴.

VAE 개념 (Encoder-Decoder)

VAE(Variational AutoEncoder)

이미지를 잠재 공간(latent space)의 **확률 분포**로 변환하여 새로운 데이터를 생성하는 모델



“ 개념 비유

Encoder: 사진을 보고 상세한 설명서(특징) 작성

Decoder: 설명서만 보고 다시 그림 그리기



기본 개념:

- VAE는 입력 데이터를 **잠재 공간(latent space)**에 확률적으로 인코딩하고, 다시 복원(decoding)하는 **생성 모델(Generative Model)**임.
- 단순한 Autoencoder와 달리, **확률 분포(μ, σ)**를 학습해 새로운 데이터 샘플을 생성할 수 있음.

AutoEncoder vs VAE

● AutoEncoder (AE)

잠재 공간이 고정값

단순 압축 및 복원 목적

새로운 생성(Sampling) 어려움

● VAE

잠재 공간이 확률 분포

데이터의 특성 분포 학습

Sampling으로 생성 가능

1. Encoder (인코더)

- 입력 데이터를 평균(μ)과 표준편차(σ)로 압축함.
- 잠재 변수 z 는 **재매개변수화 트릭(reparameterization trick)** 으로 샘플링함.

$$z = \mu + \sigma * \varepsilon (\varepsilon \sim N(0,1))$$

2. Latent Space (잠재공간)

- z 는 데이터의 **압축된 확률적 표현**으로, 새로운 데이터 생성에 사용됨.

3. Decoder (디코더)

- z 를 입력을 받아 원본과 유사한 데이터를 복원함.
- 일반적으로 **sigmoid** 활성화를 사용하여 $[0,1]$ 범위의 이미지를 출력함.

VAE Objective Function

$$\text{Loss} = \text{Reconstruction} + \text{KL Divergence}$$

Reconstruction Loss

"원본 이미지와 얼마나 비슷한가?"

입력(Input)과 복원된 출력(Output) 픽셀 간의 차이를 최소화 (MSE 등)

KL Divergence

"잠재 공간이 정규분포를 따르는가?"

잠재 변수 분포를 표준정규분포 $N(0,1)$ 에 근사시켜 생성 용이성 확보



핵심 학습 트릭: Reparameterization

역전파(Backpropagation)가 불가능한 샘플링 단계를 미분 가능하게 변경

$$z = \mu + \sigma * \epsilon \quad (\epsilon \sim N(0,1))$$

Balance & Trade-off

Reconstruction: 복원력

3

Input x Minimize Error
→

3

Recon x'

2 KL Divergence: 정규화

Target: $N(0,1)$ Learned: $N(\mu, \sigma)$ 

Insight

두 Loss가 균형을 이뤄야 잠재 공간에 빈틈이 사라지고(Continuous), 자연스러운 보간(Interpolation)과 새로운 생성이 가능해짐

구현 단계 (Step-by-Step)

PyTorch를 사용한 VAE 모델 구현 및 학습 과정입니다.

1. 데이터셋 준비

MNIST 데이터 로드 및 전처리 (ToTensor, Normalize)

2. 모델 구조 정의 (핵심)

Encoder, Decoder, 그리고 **reparameterize** 함수 구현

3. Loss 함수 설정

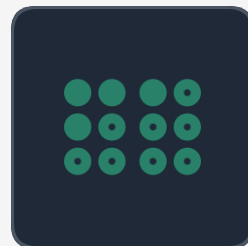
Reconstruction Loss (BCE) + KL Divergence 합산

4. 학습 루프 (Training Loop)

Forward -> Loss 계산 -> Backward -> Optimizer Step

5. 이미지 생성 (Sampling)

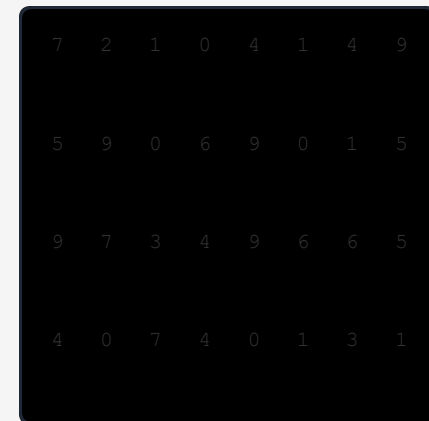
랜덤 노이즈 z 생성 후 Decoder 통과



Random Noise (z)
`torch.randn(64, 20)`



Decoder



Generated Samples

	VAE (Variational AutoEncoder)	GAN (Generative Adversarial Net)
 학습 안정성	명확한 Loss 함수로 수렴이 잘 됨	균형 깨지면 학습 실패 (Mode Collapse)
 이미지 품질	평균값을 학습하여 다소 흐릿함	디테일하고 선명한 이미지 생성
 수학적 해석	확률 분포 근사 (Likelihood)	게임 이론 & 내쉬 균형
 구현 난이도	중간 (Medium)	높음 (High) 하이퍼파라미터에 매우 민감

WRAP UP

수업 마무리 요약



생성형 모델은 "**분포(Distribution)**"를 학습한다

단순 암기가 아닌, 데이터가 생성될 확률 공간을 이해하고 새로운 샘플을 만들어냄



VAE

잠재 공간의 **확률적 분포**를 가정하고 샘플링하여 생성



GAN

생성자와 판별자의 **경쟁적 학습**을 통해 사실적 이미지 생성



활용 분야



스타일 전이

Style Transfer



이미지 생성/복원

Inpainting



데이터 증강

Augmentation